字符串

字符串的基本操作

字符串的基本操作有读入、查找、替换、截取和字符串转换等。（这里主要介绍 $String$ 类）

$string$ 类重新定义了输入输出运算符，也就是用 \\>>// 进行 $string$ 变量输入，用 \\<<//进行 $string$ 变量输出。需要注意的是，输入运算符\\>>//默认会忽略空格，一旦遇到空格就认为输入结束。如果需要输入有空格组成的一行字符，可以使用 $getline()$ 函数。如下所示：

```
getline(cin,str);   // 从cin 读入到 str 变量中
```

字符串的拼接。有了$string$类，我们可以使用 \\+// 或 \\+ =// 运算符来直接拼接字符串

提取子字符串。$substr()$函数用于从$string$ 字符串中提取子字符串，语法为：

```
string substr(pos,len);     //pos为要提取子字符串的起始下标， len 为要提取子字符串的长度，返回数据类型为字符串
```

字符串的查找。$find()$ 函数用于在$string$ 变量存储的字符串中查找子字符串出现的位置，语法为：

```
int find(str,pos);  //第一个参数str 为待查找的子字符串，第二个参数pos为开始查找的位置，如果不指明，则从第0个字符开始查找
```

运算结果返回的是子字符串第一次出现在$string$ 变量存储的字符串中的起始下标，如未找到则返回 -1 （注意输出是 -1 的补码）

P5734 文字处理软件

题目描述：你需要开发一款文字处理软件。最开始时输入一个字符串（不超过 100 个字符）作为初始文档。可以认为文档开头是第 0 个字符。需要支持以下操作：

- `1 str`：后接插入，在文档后面插入字符串 str，并输出文档的字符串。
- `2 a b`：截取文档部分，只保留文档中从第 a 个字符起 b 个字符，并输出文档的字符串。
- `3 a str`：插入片段，在文档中第 a 个字符前面插入字符串 str，并输出文档的字符串。
- `4 str`：查找子串，查找字符串 str 在文档中最先的位置并输出；如果找不到输出 -1。

为了简化问题，规定初始的文档和每次操作中的 str 都不含有空格或换行。最多会有 $q(q \leq 100)$次操作。

输入格式：无

输出格式：无

输入输出样例

输入#1	输出#1
4 ILove 1 Luogu 2 5 5 3 3 guGugu 4 gu	 ILoveLuogu Luogu LuoguGugugu 3

参考代码

```
#include<bits/stdc++.h>
using namespace std;
int q;
string s;
int main(){
    cin>>q;
    cin>>s;     //本题字符串没有空格和回车符

    while(q--){
        int x;  cin>>x;
        if(x==1){
            string st;  cin>>st;
            s = s+st;   //后面接入
            cout<<s<<endl;
        }
        if(x==2){
            int a,b;    cin>>a>>b;
            s = s.substr(a,b);
            cout<<s<<endl;
        }
        if(x==3){
            int a;
            string st;
            cin>>a; cin>>st;
            s.insert(a,st);
            cout<<s<<endl;
        }
        if(x==4){
            string st;  cin>>st;
            if(s.find(st) == -1)    cout<<"-1"<<endl;   //注意判断是否是-1 ， 直接输出会出错
            else    cout<<s.find(st)<<endl;
        }
    }
    return 0;
}
```

字符串哈希

首先 看一个比较特殊的字符串匹配问题：在很多字符串中尽快操作某个字符串。如果字符串的规模很大，访问速度很快速。这个问题用哈希（$hash$）方法解决是最快的。用哈希函数对每个子串进行哈希，分别映射到不同的数字，即一个整数哈希值，然后就可以根据哈希值找到子串，接下来配合使用数据结构或STL完成判重、统计、查询等操作。

哈希函数是其中的核心。在信息学竞赛中，最经典的就是进制哈希（$BKDRHash$函数），求得的哈希值几乎不会冲突碰撞。进制哈希的核心便是给出一个固定进制$base$，将一个串的每一个元素看做一个进制位上的数字，所以这个串就可以看做一个$base$进制的数，那么这个数就是这个串的哈希值；则我们通过比对每个串的的哈希值，即可判断两个串是否相同

P3370 【模板】字符串哈希

题目描述：如题，给定N个字符串（第 i 个字符串长度为 M_i，字符串内包含数字、大小写字母，大小写敏感），请求出 N 个字符串中共有多少个不同的字符串。

输入格式：第一行包含一个整数 N ，为字符串的个数。接下来 N 行每行包含一个字符串，为所提供的字符串。

输出格式：输出包含一行，包含一个整数，为不同的字符串个数。

输入输出样例

输入#1	输出#1
5 abc aaaa abc abcc 12345	4

算法分析

我们采用进制哈希来模拟一下本题的解题过程。

首先：进制位我们采用7进制（但是在实际问题中，我们更多采用131进制数，13331进制数）

字符对应的$ASCII$： $a=97$， $b=98$， $c=99$， $\backslash 1'=49$ ， $'2'=50$ ，$'3'=51$

$abc='c'*7^0+'b'*7^1+'a'*7^2=5538$

$abcc='c'*7^0+c'*7^1+'b'*7^2+'a'*7^3=38865$

$12345='1'*7^4+'2'*7^3+'3'*7^2+'4'*7^1+'5'*7^0=137715$

接着我们每个字符串的比较就转化成了整数的比较。将字符串的长度优化成了 1。

代码参考

```
#include<bits/stdc++.h>
using namespace std;
typedef unsigned long long ULL;
const ULL P=131;    //这个进制是经验数，也可以是13331，这样计算出来的hash值不太容易冲突
const int N=10010;
int n,ans=1;
ULL h[N];
ULL BKDRHash(char *str){
```

```
    ULL key=0;      //定义成ULL，当总数溢出时相当于做最大正整数的模运算
    while(*str){
        key = key*P + (*str++);     //转换为 P 进制 数的总和
    }
    return key;
}
int main() {
    cin>>n;
    char s[N];
    for(int i=0;i<n;i++){
        cin>>s;
        h[i] = BKDRHash(s);
    }
    sort(h,h+n);    //排序
    for(int i=1;i<n;i++){
        if(h[i] == h[i-1])  continue;
        else    ans++;
    }
    cout<<ans;
    return 0;
}
```

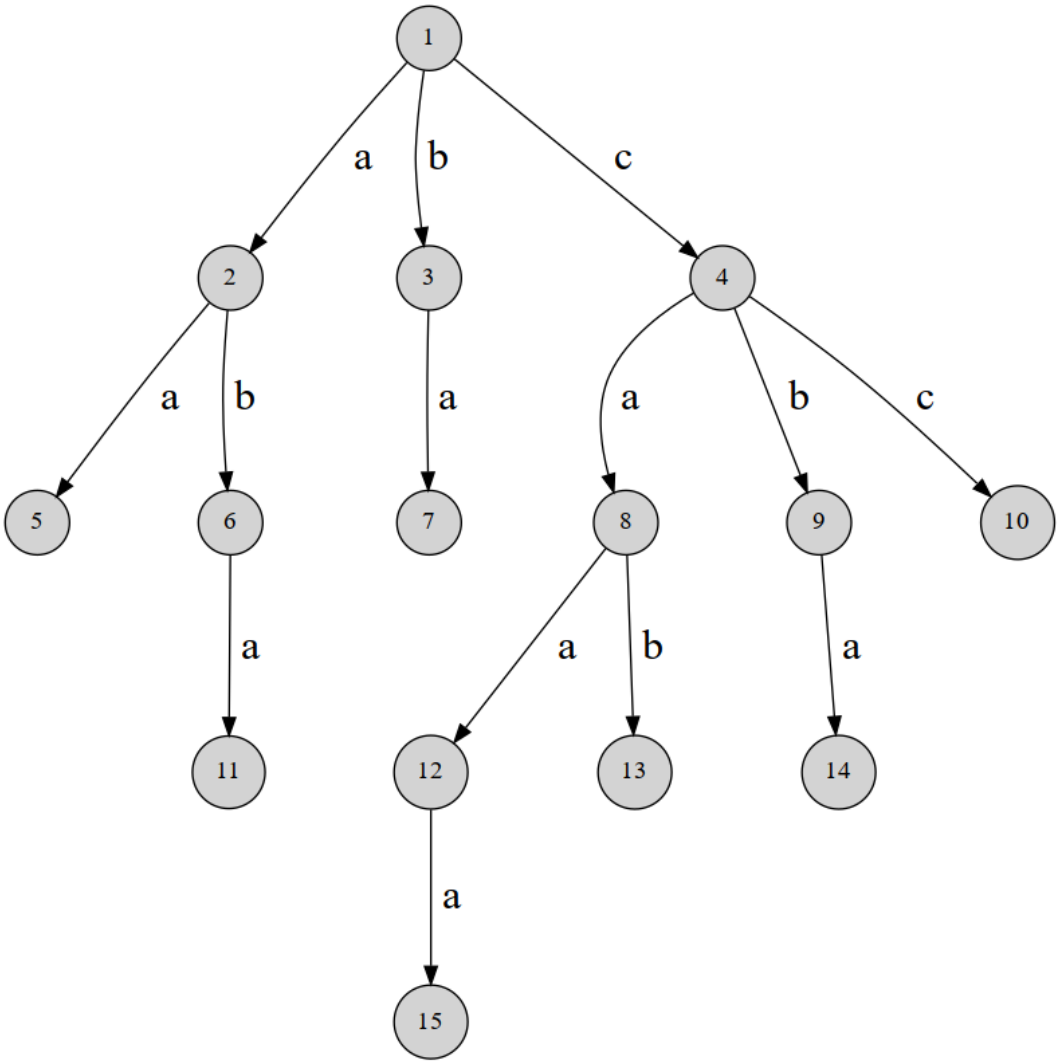
字典树（Trie 树）

有一个常见的字符串匹配问题：在n 个字符串中查找某个字符串。如果用暴力的方法，需要逐个匹配每个字符串，复杂度是$O(nm)$， m 是字符串的平均长度。这个操作的效率十分低。

日常生活中，如果需要查找单词dog，先翻到字典的 d 部分，再翻到第2 个字母 o 、第 3 个字母 g ，一共找 3 次即可。查找任意单词，查找次数最多只需要这个单词的字母的个数。

字典树就是模拟这个操作的数据结构，它的时间复杂度和空间复杂度都很好。

- 时间复杂度：插入和查找单词的复杂度都是 $O(m)$， 其中 m 是待插入/查询字符串的长度。
- 空间复杂度：有公共前缀的单词只需要存一次公共前缀，节省了空间。



Trie

可以发现，这棵字典树用边来代表字母，而从根结点到树上某一结点的路径就代表了一个字符串。举个例子，$1 \rightarrow 4 \rightarrow 8 \rightarrow 12$ 表示的就是字符串 `caa` 。

如图可以看出字典树的基本性质：根结点不包含字符，除根结点外的每个子结点都包含一个字符；从根结点到某一个结点，路径上经过的字符连接起来，为该结点对应的字符串；每个结点的所有子结点包含的字符互不相同。通常在实现的时候会在结点设置一个标志，标记该结点是否为单词的末尾。

字典树有以下常见的应用：

1. 字符串检索。检索、查询功能是字典树的基本功能。
2. 词频统计。统计一个单词出现了多少次。
3. 字符串排序。在插入的时候，在树的平级按字母表的顺序插入。字典树建立好后，用先序遍历，就得到了字典树的排序。
4. 前缀匹配。字典树是按公共前缀来建树的，很适合用于搜索提示。例如$Linux$ 的行命令，输入一个命令的前面几个字母，系统会自动补全命令后面的字符。

[模板题] hdu 1251 “字典树“

题目链接： HDU 1251 字典树

题目描述：很多单词只由小写字母组成，不会有重复的单词出现，统计出以某个字符串为前缀的单词数量。

1. 用map 实现

```
const int N=10010;
char str[N];    //单词的长度
map<string, int> m;
while(gets(str)){
    int len=strlen(str);
    if(!len)    break;  //输入了一个空行
    for(int i=len;i>0;i--){
        str[i] = '\0';  //从后往前删除这个字符串的字符，得到前缀
        m[str]++;
    }
}
while(gets(str))    cout<<m[str]<<endl;
```

2. 用数组实现

```
const int N=10010;
int trie[N][26];    //用数组定义字典树，存储下一个字符的位置
int num[N];         //以某一字符串为前缀的单词的数量。
int pos=1;          //当前新分配的存储位置
void Insert(char str[]){
    int p=0;
    for(int i=0;str[i];i++){
        int n=str[i]-'a';
        if(trie[p][n]==0)   trie[p][n]=pos++;   //新分配位置
        p=trie[p][n];   //记录该位置
        num[p]++;       //记录该前缀和的数量
    }
}
int Find(char str[]){
    int p=0;
    for(int i=0;str[i];i++){
        int n=str[i]-'a';
        if(trie[p][n]==0)   return 0;
        p=trie[p][n];   //往下找
    }
    return num[p];
}
```

字典树在后面AC自动机也有应用。

P2580 于是他错误的点名开始了

题目背景：XS中学化学竞赛组教练是一个酷爱炉石的人。他会一边搓炉石一边点名以至于有一天他连续点到了某个同学两次，然后正好被路过的校长发现了然后就是一顿欧拉欧拉欧拉（详情请见已结束比赛 CON900）。

题目描述：这之后校长任命你为特派探员，每天记录他的点名。校长会提供化学竞赛学生的人数和名单，而你需要告诉校长他有没有点错名。（为什么不直接不让他玩炉石。）

输入格式：第一行一个整数 n，表示班上人数。

接下来 n 行，每行一个字符串表示其名字（互不相同，且只含小写字母，长度不超过50）。

第 $n+2$ 行一个整数 m，表示教练报的名字个数。

接下来 m 行，每行一个字符串表示教练报的名字（只含小写字母，且长度不超过50）。

输出格式：对于每个教练报的名字，输出一行。

如果该名字正确且是第一次出现，输出 `OK`，如果该名字错误，输出 `WRONG`，如果该名字正确但不是第一次出现，输出 `REPEAT`。

输入输出样例

输入#1	输出#1
5 a b c ad acd 3 a a e	OK REPEAT WRONG

说明/提示

- 对于 40% 的数据，$n \leq 1000$，$m \leq 2000$。
- 对于 70%的数据，$n \leq 10^4$，$m \leq 2 \times 10^4$。
- 对于 100% 的数据，$n \leq 10^4$，$m \leq 10^5$。

代码参考

1. 用 map 实现

```
#include<bits/stdc++.h>
using namespace std;
const int N=10010;
int trie[N][26];    //用数组定义字典树，存储下一个字符的位置
int num[N];         //以某一字符串为前缀的单词的数量。
int pos=1;          //当前新分配的存储位置
void Insert(char str[]){
    int p=0;
    for(int i=0;str[i];i++){
        int n=str[i]-'a';
        if(trie[p][n]==0)   pos++;  //新分配位置
        p=trie[p][n];   //记录该位置
        num[p]++;       //记录该前缀和的数量
    }
}
int Find(char str[]){
    int p=0;
    for(int i=0;str[i];i++){
        int n=str[i]-'a';
        if(trie[p][n]==0)   return 0;
        p=trie[p][n];   //往下找
    }
    return num[p];
}
int main(){
```

```
    return 0;
}
```

2. 用数组实现

```
#include<bits/stdc++.h>
using namespace std;
const int N=500010; // N = n * 长度 = 10^4 * 50 = 500000
int trie[N][26];    //用数组定义字典树，存储下一个字符的位置
int num[N];         //记录点名的次数。默认为0
bool flag[N];       //标记该点为一个单词的
int pos=1;          //当前新分配的存储位置
int n,m;

void Insert(char str[]){
    int p=0;
    for(int i=0;str[i];i++){
        int n=str[i]-'a';
        if(trie[p][n]==0)   trie[p][n]=pos++;   //新分配位置
        p=trie[p][n];   //记录该位置
    }
    flag[p]=1;  //标记该前缀为结尾
}
int Find(char str[]){
    int p=0;
    for(int i=0;str[i];i++){
        int n=str[i]-'a';
        if(trie[p][n]==0)   return 0;       //表示没有这个单词，返回 wrong
        p=trie[p][n];   //往下找
    }
    if(flag[p]){
        flag[p]=0;
        return 1;   // 表示第一次点到这个名字
    }
    else if(!flag[p])   return 2;   //表示有这个名字，但是前面已经点过了
}
int main(){
    cin>>n;
    while(n--){
        char s[51]; cin>>s; Insert(s);
    }
    cin>>m;
    while(m--){
        char s[51]; cin>>s;
        int x=Find(s);
        if(x==0)    puts("WRONG");
        else if(x==1)   puts("OK");
        else if(x==2)   puts("REPEAT");
    }
    return 0;
}
```

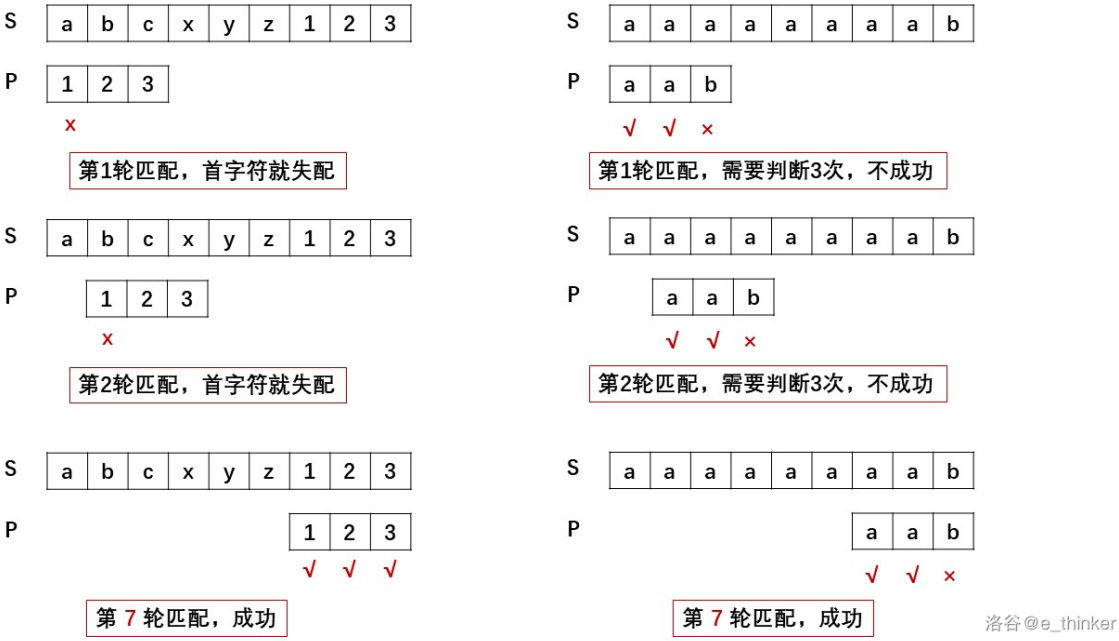
KMP 算法

KMP 算法是单模匹配算法，即在一个长度为 n 的文本串中查找一个长度为 m 的模式串。它的复杂度是 $(m+n)$，差不多是此类算法能达到的最优复杂度。

朴素的模式匹配算法（暴力方法）：在S的所有字符中逐个匹配 P 的每个字符。

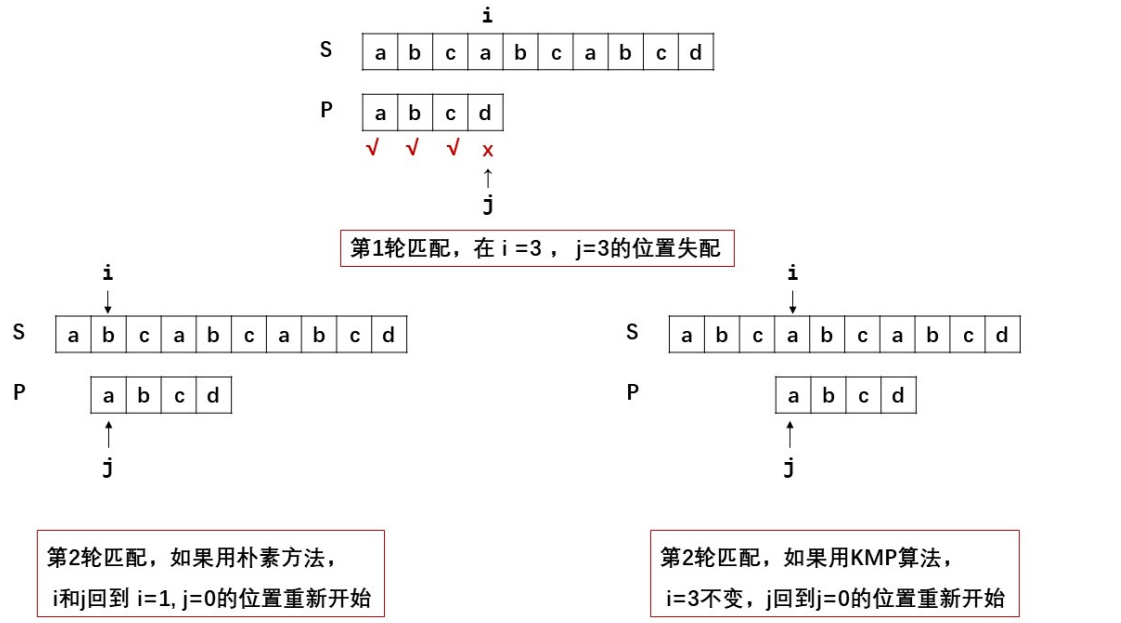
```
例1：
S = "abcxyz123"
P = "123"
```

```
例2：
S = "aaaaaaaab"
P = "aab"
```



结论：朴素算法在某种特殊字符串（比如例1）中，复杂度差不多$O(n+m)$， 这已经是字符串匹配能达到的最优复杂度了。所以，如果字符串 S , P 符合这个特征，用暴力法是不错的选择。但是，如果情况比较坏（比如例2），那么复杂度就退化成 $O(nm)$。

KMP 算法是一种在任何情况下都能达到 $O(n+m)$ 复杂度的算法。它是通过分析 P 的特征对 P 进行预处理，从而在与 S 匹配的时候能够跳过一个字符串，达到快速匹配的目的。



洛谷@e_thinker

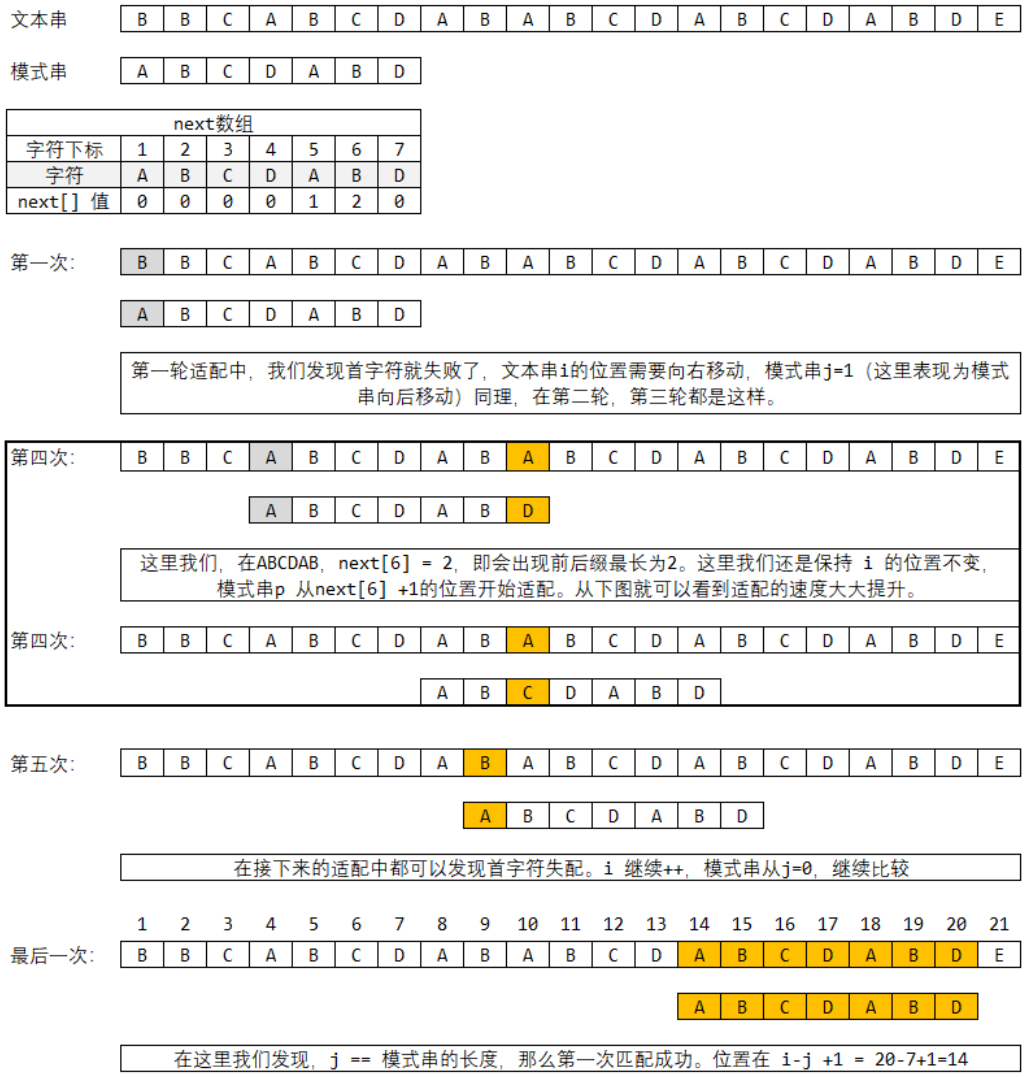
在用 KMP 算法时，指向S的 i 指针不会回溯，而是一直往后走到底。与朴素算法相比，大大减少了匹配次数。那么 KMP 是如何让 i 不回溯，只回溯 j 的呢？这就是KMP的核心——$next[\ ]$数组。当出现失配后，进行下一次匹配时，用$next[]$指出j回溯的位置。

如果给定的模式串是："ABCDABD"，从左至右遍历整个模式串，其各个子串的前缀后缀分别如下表格所示：

模式串的各个子串	前缀	后缀	最长公共元素长度
A	空	空	0
AB	A	B	0
ABC	A, AB	C, BC	0
ABCD	A, AB, ABC	D,CD,BCD	0
ABCDA	A, AB , ABC, ABCD	A, DA, CDA, BCDA	1
ABCDAB	A, AB, ABC, ABCD, ABCDA	B, AB, DAB, CDAB, BCDAB	2
ABCDABD	A, AB, ABC, ABCD, ABCDA, ABCDAB	D, BD, ABD, DABD, CDABD, BCDABD	0

字符	A	B	C	D	A	B	D
Next[ ]	0	0	0	0	1	2	0

洛谷@e_thinker



P3375 KMP算法

题目描述：给出两个字符串 s_1和s_2，若 s_1 的区间 $[l,r]$子串与s_2 完全相同，则称s_2在s_1 中出现了，其出现位置为 l 。 现在请你求出s_2 在 s_1 中所有出现的位置。

定义一个字符串 s 的 border 为 s 的一个**非 s本身**的子串 t ，满足 t 既是 s 的前缀，又是 s 的后缀。
对于 s_2 ，你还需要求出对于其每个前缀 s' 的最长 border t' 的长度。

输入格式：第一行为一个字符串，即为s_1。第二行为一个字符串，即为s_2。

输出格式：首先输出若干行，每行一个整数，**按从小到大的顺序**输出s_2 在 s_1 中出现的位置。
最后一行输出 $|s_2|$ 个整数，第i 个整数表示s_2 的长度为 i 的前缀的最长 border 长度。

输入输出样例

输入#1	输出#1
ABABABC ABA	1 3 0 0 1

代码参考

```
#include<iostream>
#include<cstring>
using namespace std;
typedef unsigned long long ull;
const int N=1000010;
int next[N];
char t[N],p[N]; //t为文本串， p为模板串
int  tl,pl,j;
int main() {
    cin>> (t+1) >> (p+1);
    tl = strlen(t+1);   pl = strlen(p+1);
    // 计算next数组(以模板串为计算）
    for(int i=2;i<=pl;i++) {
        while(j && p[i]!=p[j+1])    j=next[j];
        if(p[j+1] == p[i])  j++;
        next[i] = j;
    }
    j=0;
    //KMP算法匹配。 与next数组计算类似
    for(int i=1;i<=tl;i++) {
        while(j>0 && p[j+1]!=t[i])  j=next[j];
        if(p[j+1]==t[i])    j++;
        if(j==pl) {
            cout<<i-pl+1<<endl;
            j=next[j];
        }
    }
    for(int i=1;i<=pl;i++)  cout<<next[i]<<" ";
    return 0;
}
```

AC 自动机

AC 自动机（$Aho-Corasick\ automaton$）是KMP 的升级版。KMP 是单模匹配算法，处理在一个文本串中查找一个模式串的问题；AC 自动机是多模匹配算法，能在一个文本串中同时查找多个不同的模式串。

多模匹配问题：给定一个长度为 n 的文本S，以及k 个平均长度为 m 的模式串 P_1 ， P_2 ， $\ldots$ ，P_k ，要求搜索这些模式串出现的位置。

其实用KMP 也能解决多模匹配问题，缺点是复杂度较高，需要对每个 P_1 ， P_2 ， $\ldots$ ，P_k 分别做一次 KMP，总复杂度是$O((n+m)k)$。

AC 自动机算法并不需要对 S 做多次 KMP，而是只搜索一遍 S，在搜索时匹配所有的模式串。如何同时匹配所有的P？结合前面的字典树就好理解了。KMP是通过查找 P 对应的 $next[\ ]$数组实现快速匹配的。如果把所有的 P 做成一个字典树，然后在匹配的时候查找这个 P 对应的 $next[\ ]$ 数组，不就实现了快速匹配的效果吗?

复杂度分析：k 个模式串，平均长度为 m；文本串长度为 n。建立字典树 $O(km)$；建立 $fail$ 指针 $O(km)$；模式匹配 $O(nm)$，乘 m 的原因是在统计的时候需要顺着链回溯到 $root$ 结点。总时间复杂度是$O(km+km+nm)=O(km+nm)$。

对比简单使用 KMP 的复杂度 $O((n+m)k)$，当$m<<k$时，$(k+n)m<<(n+m)k$。AC 自动机优势非常大。

P3808 【模板】AC自动机（简单版）

题目描述：给定 n 个模式串 s_i和一个文本串 t ，求有多少个不同的模式串在文本串里出现过。两个模式串不同当且仅当他们**编号**不同。

输入格式：第一行是一个整数，表示模式串的个数 n 。第 2到第 $(n+1)$ 行，每行一个字符串，第 $(i+1)$行的字符串表示编号为i的模式串s_i 。最后一行是一个字符串，表示文本串t 。

输出格式：输出一行一个整数表示答案。

输入输出样例：

输入#1	输出#1	输入#2	输出#2	输入#3	输出#3
3 a aa aa aaa	3	4 a ab ac abc abcd	3	2 a aa aa	2

说明/提示：

- 样例 1 解释：s_2 与s_3 编号（下标）不同，因此各自对答案产生了一次贡献。
- 样例 2 解释：s_1，s_2，s_4 都在串 `abcd` 里出现过。

数据规模与约定：

- 对于 50% 的数据，保证$n=1$。
- 对于100% 的数据，保证 $1 \leq n \leq 10^6$，$1 \leq |t| \leq 10^6$，$1 \leq \sum_{i=1}^{n} |s_i| \leq 10^6$。

代码参考

```
#include<bits/stdc++.h>
using namespace std;
const int N= 1e6+10;
int n;
int tr[N][26],tot;
int e[N], fail[N];
void insert(char *s){   //建立字典树
    int u=0;
    for(int i=1;s[i];i++){
        if(!tr[u][s[i]-'a'])    tr[u][s[i]-'a']=++tot;
        u=tr[u][s[i]-'a'];
    }
    e[u]++;
}
queue<int> q;

void build(){   //创建失配指针
    for(int i=0;i<26;i++){
        if(tr[0][i])    q.push(tr[0][i]);
    }
    while(q.size()){
        int u=q.front();
        q.pop();
        for(int i=0;i<26;i++){
```

```
            if(tr[u][i]){
                fail[tr[u][i]] = tr[fail[u]][i];
                q.push(tr[u][i]);
            }
            else    tr[u][i]=tr[fail[u]][i];
        }
    }
}

int query(char *s){     //多模匹配
    int u=0,res=0;
    for(int i=1;s[i];i++){
        u=tr[u][s[i]-'a'];
        for(int j=u; j&& e[j]!=-1; j=fail[j]) {
            res +=e[j];
            e[j]=-1;
        }
    }
    return res;
}
char s[N];
int main(){
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>s+1;
        insert(s);
    }
    cin>>s+1;
    build();
    cout<<query(s);
    return 0;
}
```